

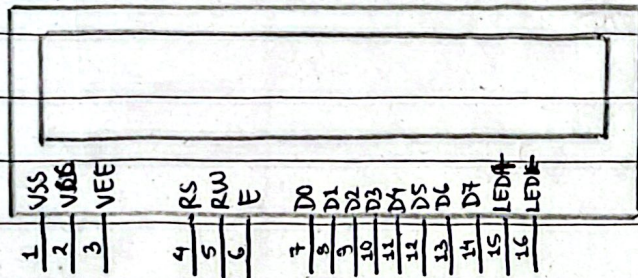
Unit-6 Peripherals & Interfacing

display fixed patterns (alarm), (can display image, custom characters)

LCD (Used to display number, text, graphics. Types of LCD: Segment LCD, Graphic LCD, (character LCD (16x2).

LCD 16x2 Display

- Two rows each capable of displaying 16 characters
- 16 pins, LCD works by controlling liquid crystals to block or pass light. Forms character using dot matrix



No.	Pin	Function
1	VSS	Ground
2	VCC	+5V Supply
3	VEE	Display contrast control (controlled by varying voltage by potentiometer) GND → max contrast
4	RS	Register select (0 = command Register, 1 = Data Register)
5	RW	Read/Write (0 = write, 1 = Read)
6	E	Enable
7-14	D0-D7	Data Pins (Parallel data/command interface; 4 bit mode ⇒ D4-D7 used)
15	LED+	ve supply for backlight
16	LED-	-ve supply for backlight

Register select

RS = 0 : The data sent to D0-D7 is interpreted as command
RS = 1 : The data sent is interpreted as content to be displayed

Enable

This pin is used to tell LCD to read data from data pins.

RW

RW = 0 : Data written to LCD
RW = 1 : Data read from LCD

Q) Write an AVR C program to display the message "Hello, World!" on a 16x2 LCD using 4 bit mode.

```
#define F_CPU 16000000UL
```

```
#include <avr/io.h>
```

```
#include <util/delay.h>
```

```
#define LCD_PORTA
```

```
#define EN 7
```

```
#define RW 6
```

```
#define RS 5
```

```
void lcdcmd(unsigned char cmd)
{
```

```
    PORTD &= ~(1<<RS);           //RS=0 for command
```

```
    PORTD &= ~(1<<RW);           //RW=0 for write
```

```
    LCD = cmd;
```

```
    PORTD |= (1<<EN);            //Enable Pulse
```

```
    _delay_ms(1);
```

```
    PORTD &= ~(1<<EN);
```

```
    _delay_ms(3);
```

```
}
```

```
void lcddata(unsigned char data)
```

```
{
```

```
    PORTD |= (1<<RS);           //RS=1 for data
```

```
    PORTD &= ~(1<<RW);           //RW=0 for write
```

```
    LCD = data;
```

```
PORTD |= (1 << EN);
~delay-ms(1);
PORTD &= ~(1 << EN);
~delay-ms(3);
}
```

void lcd-init()

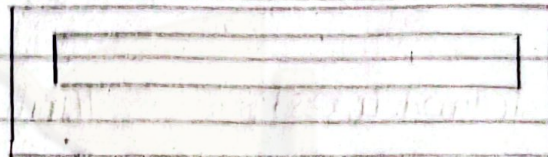
```
{
    DDRA = 0xFF; //define output port
    DDRD = 0xFF; //define RS, EN, RW pin as o/p
    PORTD &= ~(1 << EN); // Initialize EN = 0

    lcdcmd(0x38); //Initialization of 16x2 LCD in 8 bit mode
    lcdcmd(0x0C); //display on cursor off.
    lcdcmd(0x06);
    lcdcmd(0x01);
    lcdcmd(0x80);
}
```

void lcd-print(char *str)

```
{
    unsigned char i=0;
    while(str[i] != 0)
    {
        lcddata(str[i]);
        i++;
    }
}
```

```
int main()
{
    lcd-init();
    lcd-print("Hello Arpan");
    lcdcmd(0x00); // Go to 2nd line
    lcd-print("second line");
    return 0; while(1);
    return 0;
}
```



Peripherals are external components that allow ^{microcontroller} to sense, process & act upon i/p & o/p.
adrikari-arpan/BECE Notes
 # Sensors allow system to gather information about the environment.

Date _____
 Page _____

Analog sensor

output Analog output which is continuous voltage between 0 to 5V mostly
 signal type smooth
 ADC Required Yes
 Accuracy Higher (detailed values)
 Complexity More complex
 Example LM35 (Temp^r sensor), LDR

Digital sensor

Binary output (either HIGH or LOW corresponding to 1 or 0)
 Discrete
 No
 Lower (basic detection)
 Easier to use
 DHT11 (Temp^r and Humidity sensor),
 PIR (motion sensor)

LM35 (Temperature sensor)

- Analog sensor used to measure temperature in °C.

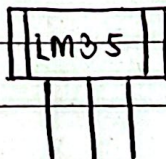
Working:- LM35 gives analog voltage proportional to 10mV per °C.

e.g. 0°C → 0V

25°C → 0.25V

$$\text{So, Temperature (}^{\circ}\text{C)} = \frac{\text{Output voltage} \times 100}{10\text{mV}}$$

$$\text{Temp}^r = \frac{\text{O/p voltage}}{10\text{mV}}$$



1 2 3
 V_{cc} V_{out} GND

1 → V_{cc}

2 → V_{out}

3 → GND

$$\text{Temp}^r = \frac{\text{O/p voltage}}{10\text{mV}}$$

- Linear output (easy calculation)
- No calibration needed
- Low power consumption
- High accuracy ($\pm 0.5^{\circ}\text{C}$)

- wide Range: -55°C to 150°C

- Works with 4V-30V supply

Interface with Atmega 32:

LM35 Pin

Atmega 32

V_{cc}

5V

V_{out}

ADC

GND

GND

a) Program to interface temperature sensor

```
#define F_CPU 8000000UL
```

```
#include <avr/io.h>
```

```
void adc-init() {
```

```
    DDRA = 0x00; //Input
```

```
    ADMUX = 0xE0;
```

```
    ADCSRA = 0x87;
```

```
}
```

```
unsigned char adc-read() {
```

```
    ADCSRA |= (1 << ADSC); //start conversion
```

```
    while (ADCSRA & (1 << ADSC)); //wait for end of conversion
```

```
    return ADCH;
```

```
}
```

```
int main(void) {
```

```
    DDRB = 0xFF; //Output
```

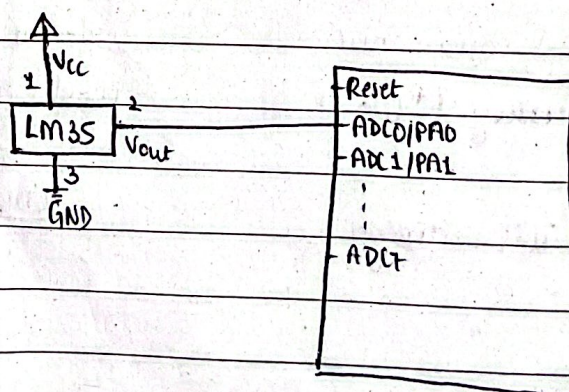
```
    adc-init();
```

```
    while (1) {
```

```
        PORTB = adc-read();
```

```
    }
```

```
}
```



Assembly program:

.include "m16def.inc"

ldi r16, 0x00 (Load Immediate)
out DDRA, r16ldi r16, 0xE0
out ADMUX, r16
ldi r16, 0x87
out ADCSRA, r16
ldi r16, 0xFF
out DDRB, r16

main_loop:

sbi ADCSRA, ADSC (set bit)

wait_adc:

sbic ADCSRA, ADSC

rjmp wait_adc ;relative jump

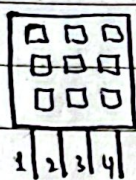
in r16, ADCH

out PORTB, r16

rjmp main_loop

Temperature & Humidity Sensor (DHT11), DHT22

- Measure temp^r and amount of water vapor present in air
- Digital sensor
- Use single digital signal to provide data for both temp^r and humidity
- Need a pull-up resistor to ensure proper communication
- cheap, easy to use, popular for Arduino, AVR project



4 pins: 1 → V_{CC}
2 → Data (connect to pull up resistor)
3 → NC (Not connected)
4 → GND

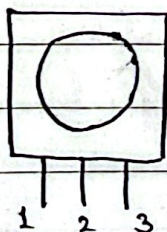
Range: 0-50°C (Temp^r)
20-90% RH (Humidity)

Supply: 3.3-5V

Update rate: ~1 Hz (Read once per second)

PIR Sensor / (Passive Infrared) Motion sensor

- Digital sensor
- Detects motion by sensing infrared radiation emitted by humans or animals
- "Passive" means it doesn't emit anything, it just detects IR from objects
- Common for security systems, automatic lights, alarms



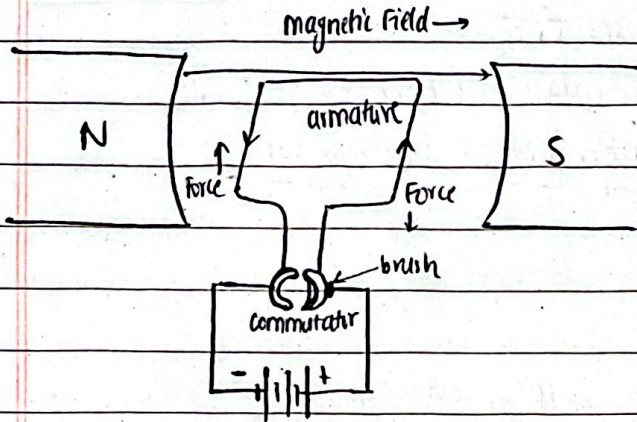
3 pins:
1 → V_{CC}
2 → ~~OUT~~ OUT
3 → GND

Interface:	Sensor Pin	AVR Pin	
	V _{CC}	+5V	Input pin → read digital value
	OUT	PDO	If HIGH → motion detect → trigger LED, buzzer or alert
	GND	GND	If LOW → no motion

Actuator Interfacing

Actuator are device that perform actions in response to command from microcontroller.

DC Motor :

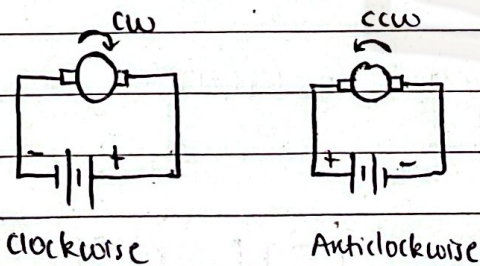


Setup as given. When current is passed in coil, by using Fleming Left hand rule, force is exerted as shown in fig. Cause armature to rotate on clockwise direction.

Controlling DC Motor includes: (i) controlling the motor's speed
(ii) controlling motor direction of rotation (cw or ccw).

(i) controlling direction

Simple Method (by polarity)



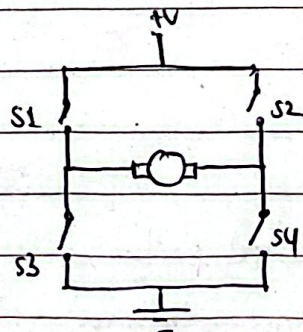
Unidirectional control → rotates in one direction based on polarity

bidirectional control → Reversing direction

(i) Relays

(ii) H-bridge circuit

H-bridge



cw → S1 & S4

ccw → S2 & S3

All off → stop

Motor driver IC (L298N Motor driver)

- Dual H-bridge IC
- can control 2 DC motors or 1 stepper motor
- Use motor drives because :
 - Back EMF (don't connect directly to μc)
 - Insufficient power
 - High current damage to μc

(ii) controlling speed

Method 1: Variable voltage (Increase voltage $\rightarrow$ Increase speed, Not practical)
More loss.

Method 2: Pulse width modulation (PWM)

≠ PWM for controlling brightness, speed

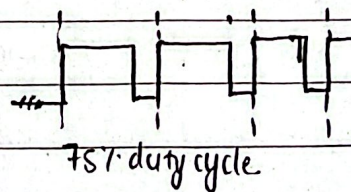
PWM is a technique used to generate analog like output using digital signals, so that analog device can operate using digital.

Concept: Duty cycle: percentage of how long signal stay HIGH in a cycle.

$$\text{Duty cycle} = \frac{T_{ON}}{T_{ON} + T_{OFF}} \times 100\%$$

T_{ON} = High

T_{OFF} = Low



PWM working:

- Signal rapidly switches ON and OFF
- Frequency is so high (human eye can't detect flicker)
- Device works on avg. voltage

Switching frequency $\rightarrow$ freq. of switch

$$V_{avg} = \text{duty cycle} \times V_{max}$$

controlling brightness :

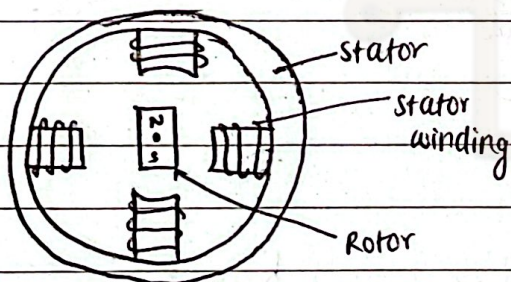
- LED bulb turns ON and OFF very fast.
- Human eye can't detect flicker.
- Works on avg. voltage
- Low duty cycle $\rightarrow$ DIM light
- High duty cycle $\rightarrow$ Bright light

controlling speed :

- Controls speed by varying average voltage supplied to motor.
- Instead of continuous voltage, rapidly switching voltage.
- Speed depends on duty cycle. $V_{avg} = \text{duty cycle} \times V_{max}$
- At Off, keeps moving due to inertia. SO, act like working with avg. Voltage (less speed)

Stepper Motor

- Convert electrical signal to discrete mechanical steps.
- Unlike DC motor, stepper move in fixed step, not continuous rotation. Provide precise position control.



1. Current flows through stator winding $\rightarrow$ creates magnetic field.
2. Rotor aligns with magnetic field.
3. Next coil energized. $\rightarrow$ rotor move to next position.
4. Repeat steps.

motor driver circuit used for speed control, direction control

Stepper Motor driving modes :

1. Wave Drive (Single coil Excitation)
2. Full step drive (Two coil Excitation)
3. Half step drive (combination)

A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ Repeat

A+B $\rightarrow$ B+C $\rightarrow$ C+D $\rightarrow$ D+A $\rightarrow$ Repeat

A $\rightarrow$ A+B $\rightarrow$ B $\rightarrow$ B+C $\rightarrow$ C $\rightarrow$ C+D $\rightarrow$ D $\rightarrow$ D+A $\rightarrow$ Repeat
(Rotor move in smaller step, more precision)

Step angle = Basic movement of stepper (angle in each step)

$$\text{step angle} = \frac{N_s - N_r}{N_s \times N_t} \times \frac{360}{N \times P}$$

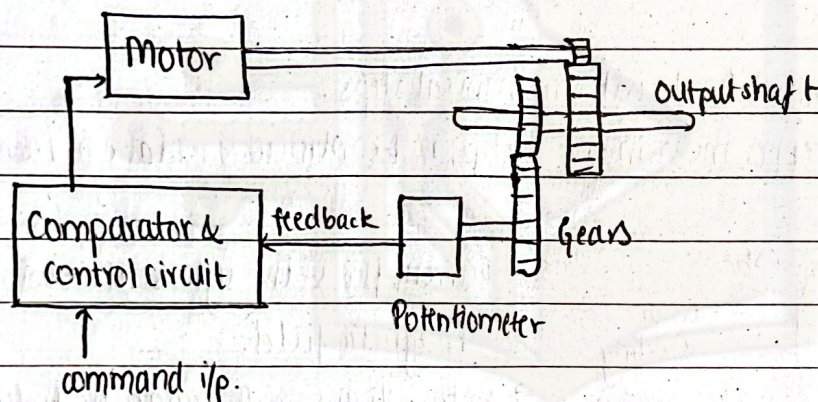
N_s → No. of stator pole
 N_r → No. of rotor pole

N → No. of teeth of rotor, P → No. of phases.

$$\text{Steps per revolution} = \frac{360^\circ}{\text{step angle}}$$

Servo Motor : standard servo: 0 to 180° rotation

- A servo motor is a rotary actuator that provides precise control of position, speed and sometimes torque.
- Used to complete task in precise way
- Two types of servo motor: (i) AC servo motor (ii) DC servo motor



- Servo Motor works using closed loop control system with feedback.

Components: 1. Motor (AC or DC)

2. Feedback Device (Potentiometer or Encoder)

3. Control circuit (Brain of system, compares desired & current condition)

4. Drive circuit (Provides power to motor, controls direction & speed)

5. Gear Box

Working (Closed loop system):

- We give input (desired angle) by PWM.
- controller read current position from sensor.
- Compare value. $\text{Error} = \text{Desired Position} - \text{current Position}$
- If error exists, motor rotates.
- Feedback updates continuously.
- Motor stop when error = 0.

{ continuous correction $\Rightarrow$ closed loop control }

- PWM in servo is used to control angle. (Not speed but angle).

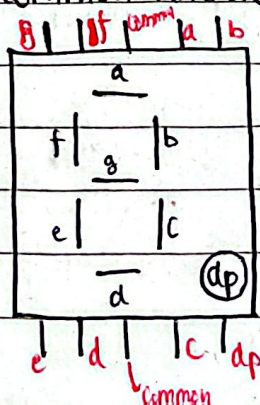
E.g. ~~~1ms~~ Pulse width: $\sim 1\text{ms} = 0^\circ$
 $\sim 1.5\text{ms} = 90^\circ$

- Used in robotic arms, camera positioning

Interface		
Red wire	Vcc	MC 5V
Black wire	GND	GND
Yellow/orange	Control signal	PWM pin

Display Interfacing (Display data in human readable form)# Seven segment display

- Electronic device used to represent number & limited alphabetic characters.
- consists of seven LED segments (labeled 'a' to 'g'), with an additional dot segment (dp) for decimal points.
- characters displayed by selectively illuminating these segments.
- Types: 1. Common Cathode (Common pin = GND, Illumination by supplying (logic 1) V)
 2. Common Anode (Common pin = Vcc, supply (logic 0) for illumination)



Q) Display from 0 to 9.

```
#define F_CPU 8000000UL
```

```
#include <avr/io.h>
```

```
#include <util/delay.h>
```

```
int main(void){
```

```
    DDRC = 0xFF; //set PortC as o/p
```

```
    PORTC = 0x00; //Turn off all segments initially
```

```
    char array[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07,  
                    //Hex values from 0 to 9          0x7F, 0x6F }
```

```
    while(1){
```

```
        for(int i = 0; i < 10; i++){
```

```
            PORTC = array[i];
```

```
            _delay_ms(1000);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

increment on button press:

```
#define SWITCH PIN0 PD0
```

```
#define SWITCH_PIN PIN0
```

```
#define SWITCH_PIN
```

```
DDRC = 0xFF;
```

```
PORTC = 0x00; //Initial 0
```

```
DDRD = ~(1 << SWITCH);
```

```
PORTD = (1 << SWITCH);
```

```
//Pull up resistor
```

```
char char array[] = { ... }
```

```
int count = 0;
```

```
while(0){
```

```
    if(!(SWITCH_PIN & (1 << SWITCH)))
```

```
    {
```

```
        count++;
```

```
        if(count > 9)
```

```
            count = 0;
```

```
        PORTC = array[count];
```

```
    }
```